21 · Concurrency, Request Rate & Arrival Patterns

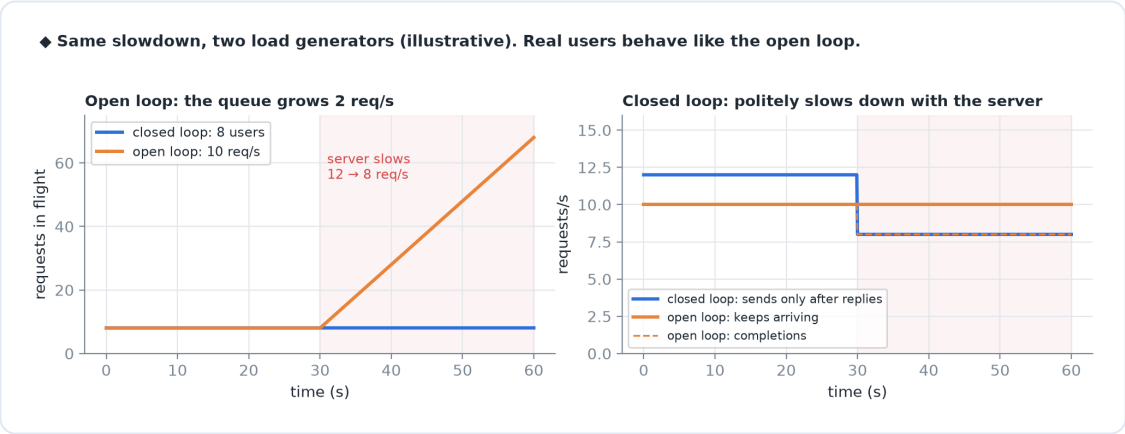
Source: fanout.sh / inference-eng / 04-02 · ~6 min

IN SHORT

How you generate load (how many at once, how many per second, and exactly when they arrive) **changes every number you measure**, so two teams can report great TTFT and terrible tails on the same server. A **closed loop** (fixed concurrency) slows down with the server; an **open loop** (fixed rate) keeps arriving and reveals the queue real users feel. **Little's law** links them: in flight = rate × time in system (4 req/s × 6 s = **24**). At the same 10 req/s on a server 80% busy, P99 wait goes **0 → 0.9 s → 2.7 s** from uniform to Poisson to bursty arrivals. Time in system grows as 1 ÷ (1 − utilization): **2×, 5×, 10×, 20×** at 50, 80, 90, 95%. That's the knee.

1. Closed loop vs open loop

- **Fixed concurrency (closed loop):** e.g. **8 virtual users**, each sends, waits for the answer, then immediately sends again. Always exactly 8 in flight.
- **Request rate (open loop):** requests arrive on a schedule, e.g. **10 per second**, whether or not earlier ones finished.



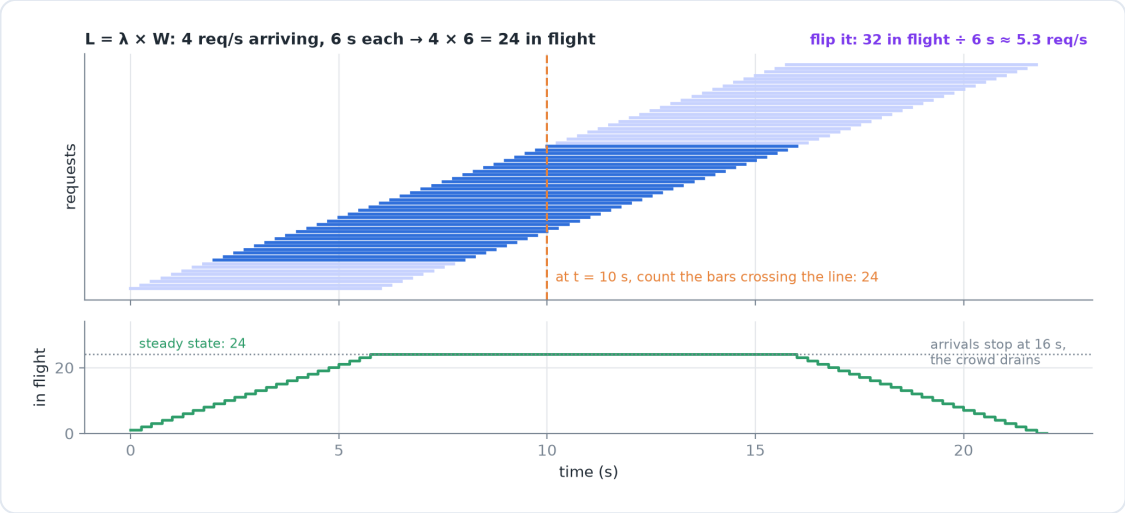
When the server slows, the closed loop sends less (its users are waiting), so nothing piles up. The open loop keeps arriving and the queue grows.

KEY IDEA

Real internet users behave like the **open loop**. A closed-loop test can hide exactly the queueing your users will feel.

2. Little's law

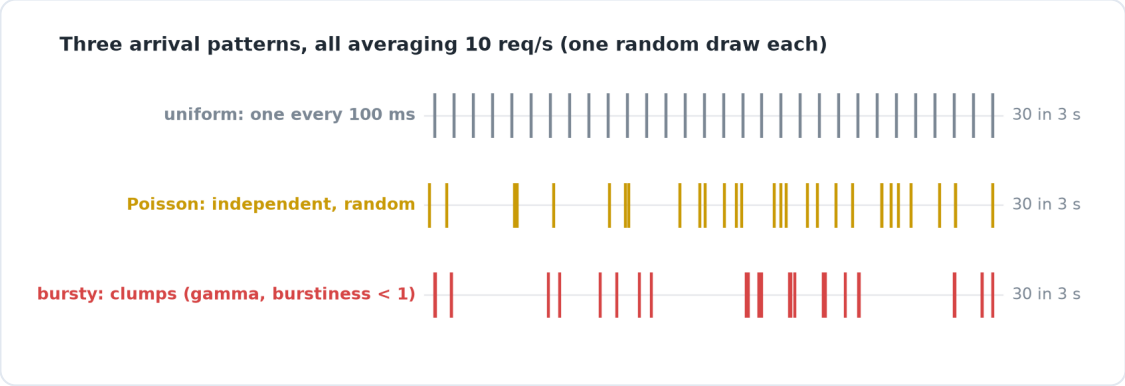
$$L = \lambda \times W \qquad \text{in flight} = \text{arrival rate} \times \text{time in system}$$



4 requests arrive per second and each takes 6 s end to end, so 24 are in flight on average. Flip it: 32 in flight at 6 s each means ≈ 5.3 req/s served.

- It **translates between** a concurrency test and a rate test.
- It warns you: if requests **arrive faster than they finish**, time in system keeps growing, and so does the crowd.

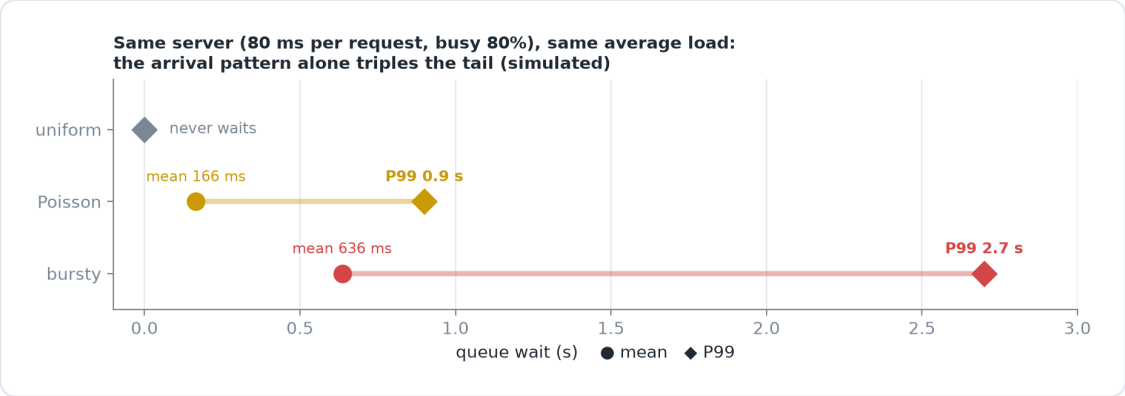
3. Arrival patterns



Same average, different timing.

Pattern	Looks like	Models
Uniform	One every 100 ms, a metronome	Almost no real traffic
Poisson	Independent, random: pairs and gaps	Many independent users
Bursty	Clumps	Batch jobs, a feature going viral; benchmark tools use a **gamma distribution with burstiness < 1**

Feed all three into the same simulated server: **80 ms per request at 10 req/s, so busy 80%** of the time.



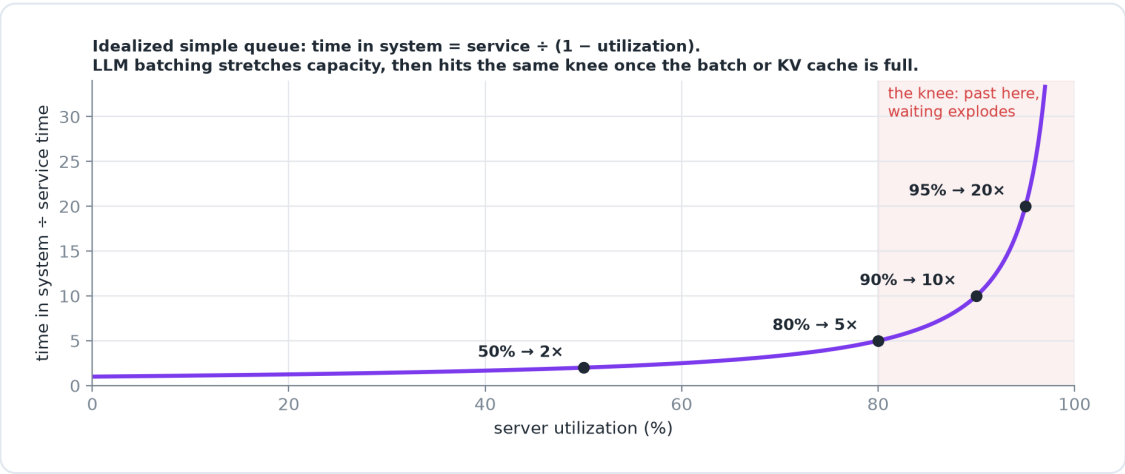
Uniform never waits. Poisson: mean 166 ms, P99 0.9 s. Bursty: mean 636 ms, P99 2.7 s.

KEY IDEA

Same average load: **the pattern alone changed the tail by 3×**.

4. The knee

In a simple queue, time in system relative to the bare service time grows as 1 ÷ (1 − utilization).



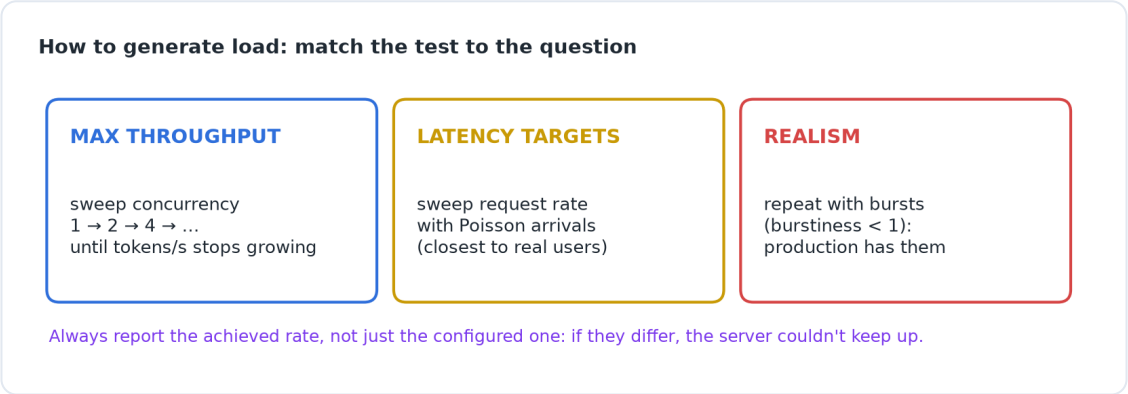
Gentle at half load, vertical near full load.

- **LLM servers batch**, so capacity stretches as load grows, but they hit the **same knee** once the batch or the **KV cache is full**.

CAUTION

A GPU at 99% busy isn't efficient: it's a queue waiting to explode.

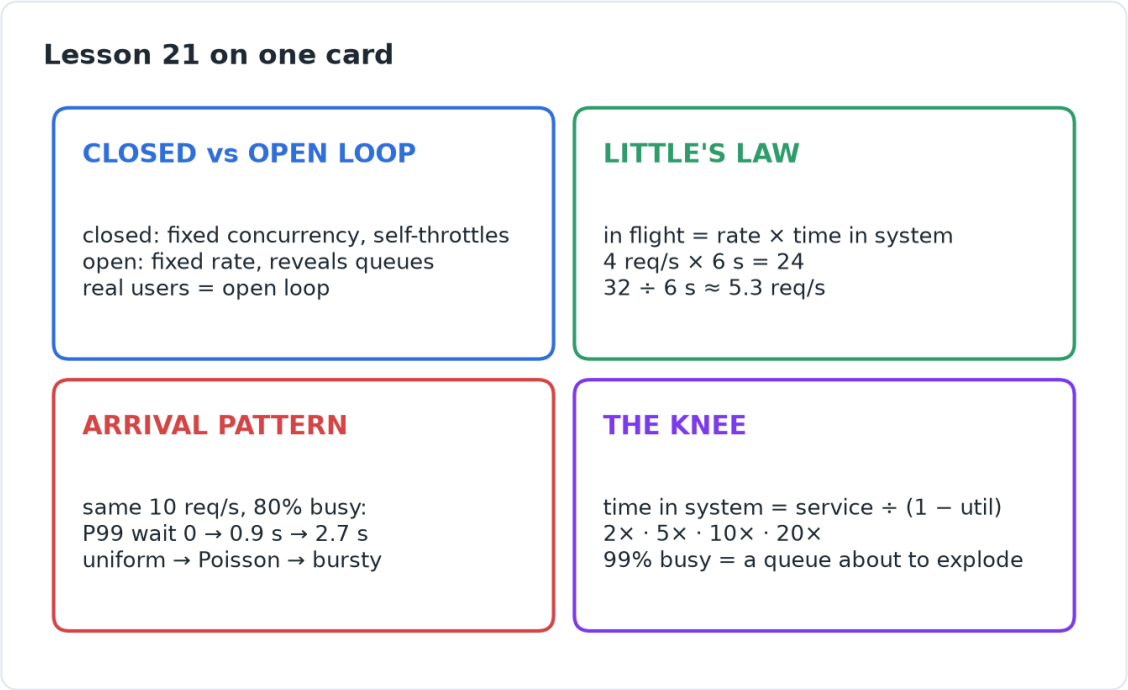
5. How to generate load



vLLM's benchmark exposes exactly these knobs:

```
vllm bench serve \
  --request-rate 10 \
  --burstiness 0.5 \
  --max-concurrency 64
```

6. Recap



Interview cheat sheet

Revise from these pages alone. ◆ = my addition beyond the video; everything else is from the lesson. Percentiles and noise are in 06; TTFT/TPOT/E2E definitions in 20.

Say it in 30 seconds

> *Load shape changes every number. A closed loop fixes concurrency: users wait for replies, so a slow server just gets fewer requests and queues stay hidden. An open loop fixes the arrival rate like real users, so queues show. Little's law ties them: in flight equals rate times time in system, 4 per second at 6 seconds is 24. At the same average rate, burstier arrivals give far worse tails: on a server 80% busy, P99 wait went from 0 to 0.9 to 2.7 seconds for uniform, Poisson and bursty. And time in system scales like 1 over 1 minus utilization, 5× at 80% and 20× at 95%, so sweep concurrency for max throughput, sweep Poisson rate for latency targets, add bursts, and report the achieved rate.*

Only numbers worth memorizing

- **Knee:** 1 ÷ (1 − ρ) = **2× / 5× / 10× / 20×** at 50 / 80 / 90 / 95% busy.

Derive, don't memorize

Let λ = arrival rate, W = time in system, S = service time, ρ = utilization.

1. Little's law: in flight = rate × time in system

Every request occupies a slot for W seconds and λ new ones show up each second, so λW are inside at any moment.

$$L = \lambda W = 4 \times 6\ \text{s} = 24 \qquad \lambda = \frac{L}{W} = \frac{32}{6\ \text{s}} \approx 5.3\ \text{req/s}$$

> **INTUITION**
>
> Converts a concurrency sweep into a rate and back. ◆ If W rises with load, the same concurrency serves fewer req/s.

2. Utilization = rate × service time

$$\rho = \lambda S = 10\ \text{req/s} \times 80\ \text{ms} = 0.8$$

3. The knee: time in system = S ÷ (1 − ρ)

The busier the server, the likelier a new request finds someone ahead of it, and the line feeds on itself.

$$\frac{W}{S} = \frac{1}{1-\rho}: \quad 0.5 \Rightarrow 2 \quad 0.8 \Rightarrow 5 \quad 0.9 \Rightarrow 10 \quad 0.95 \Rightarrow 20$$

> **INTUITION**
>
> Halving the idle time (10% → 5%) doubles the time in system. ◆ At 99% busy it's 100×.

4. Poisson wait with a fixed service time ◆

Random arrivals into a server with a constant 80 ms per request (an M/D/1 queue) wait on average:

$$W_q = \frac{\rho}{2(1-\rho)} \cdot S = \frac{0.8}{0.4} \times 80 \text{ ms} = 160 \text{ ms}$$

INTUITION

◆ Matches the lesson's simulated 166 ms. Random service times (M/M/1) double it to 320 ms; burstier arrivals push it higher still.

Closed vs open loop

	Closed loop	Open loop
You fix	Concurrency (e.g. 8 users)	Arrival rate (e.g. 10 req/s)
Server slows down	Fewer requests sent: self-throttles	Same arrivals: queue grows
Hides	Queueing delay	Nothing: shows it
Use for	Max throughput (sweep 1, 2, 4, …)	Latency targets (Poisson, then bursts)

Symptom → diagnosis

Symptom	Diagnosis
Two teams, same server: great TTFT vs terrible tails	Different load shapes (closed vs open, uniform vs bursty)
Benchmark latency fine, production TTFT bad	Closed-loop test hid the queue; prod is open and bursty
Achieved rate below configured rate	Server couldn't keep up: past capacity
Latency fine at 70%, explodes at 90%	The knee: 1 ÷ (1 − ρ)
Mean wait OK, P99 awful at the same load	Bursty arrivals
LLM server fine, then sudden latency cliff	Batch or KV cache full: batching stopped stretching capacity

Rapid-fire Q&A

Question	Crisp answer
Which loop matches real users?	Open: they don't wait for each other.
Little's law?	L = λW (derivation 1).
Why not run GPUs at 99% busy?	Time in system ≈ 100× service ◆ (derivation 3).
How do tools model bursts?	Gamma arrivals with burstiness below 1 (Poisson = 1 ◆).
What to report besides config?	The achieved request rate.

TRAP

- Quoting latency without the load shape (concurrency or rate, and arrival pattern).
- Using only closed-loop tests to set latency SLOs.
- Treating high utilization as efficiency.